

Experiment 6-Artificial Neural Network (ANN)

Objective

Implementing artificial neural networks (ANNs) without backpropagation involves creating a model that can learn from data and make predictions.

Theory

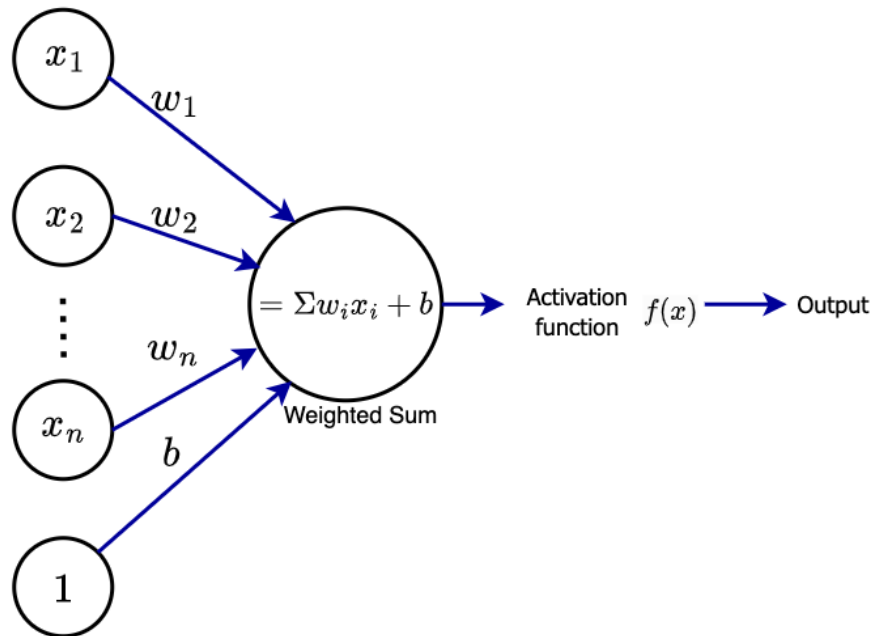


Figure 1: Structure of a Single-Layer Artificial Neural Network

Code

```
import numpy as np
import matplotlib.pyplot as plt

# Threshold activation function
def activation_function(x):
    return np.where(x >= 0, 1, 0)

# Input data (linearly separable problem)
X = np.array([[0, 0],
              [0, 1],
              [1, 0],
              [1, 1]])

# Desired output
y = np.array([0, 0, 0, 1])

# Initialize weights and bias
np.random.seed(0)
weights = np.random.rand(2)
bias = np.random.rand()

learning_rate = 0.1
epochs = 100

# Training the single-layer ANN
for epoch in range(epochs):
    for i in range(len(X)):
        net_input = np.dot(X[i], weights) + bias
        output = activation_function(net_input)

        # Error calculation
        error = y[i] - output

        # Weight and bias update
        weights = weights + learning_rate * error * X[i]
        bias = bias + learning_rate * error

# Display trained parameters
print("Final Weights:", weights)
print("Final Bias:", bias)

# Testing the ANN
print("\nANN Predictions:")
for i in range(len(X)):
```

```

        net_input = np.dot(X[i], weights) + bias
        print(X[i], "->", activation_function(net_input))

# Plot decision boundary
x1 = np.linspace(-0.2, 1.2, 100)
x2 = -(weights[0] * x1 + bias) / weights[1]

plt.figure()
plt.scatter(X[:, 0], X[:, 1])
plt.plot(x1, x2)
plt.xlabel("Input x1")
plt.ylabel("Input x2")
plt.title("Decision Boundary of Single-Layer ANN")
plt.show()

```

Output

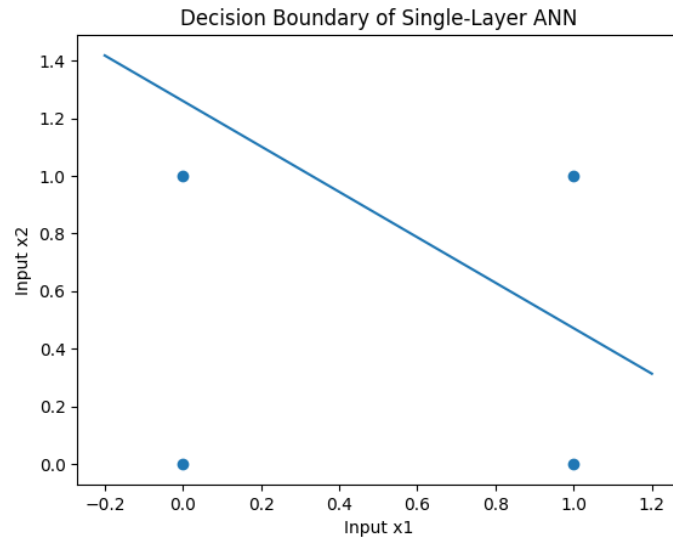


Figure 2: Decision boundary of the trained Artificial Neural Network

Final Weights = $[0.2488135 \quad 0.31518937]$

Final Bias = -0.3972366239

ANN Predictions:

[0 0] -> 0
[0 1] -> 0
[1 0] -> 0
[1 1] -> 1

Conclusion

Successfully implemented